

PYTHON CRYPTOGRAPHY & SECURITY SCRIPTING

Ultimate Infinity-Grade Reference Cheat Sheet

⚡ CORE DOMAINS & TOPICS COVERED

- Hashing & File Integrity
- PBKDF2 Key Derivation
- HMAC & Verification
- Secure Randomness (secrets)
- AES-CBC vs AES-GCM (AEAD)
- RSA Encryption & Signatures
- Encoding, XOR & Packing
- Network Automation & Pwntools
- Modular Arithmetic & RSA Math
- Itertools (Perms/Combs)
- Multi-Threading & ThreadPool
- Security Best Practices

SECTION 1 — HASHLIB & BASIC HASHING

Hashing converts input data of arbitrary size into a fixed-length digest. Cryptographic hashes are *one-way*, *deterministic*, subject to the *avalanche effect* (small input change produces vastly different digest), and have a *fixed output size*.

⚠ Security Warning: MD5 (128-bit) and SHA-1 (160-bit) are cryptographically broken for collision resistance. Do not use them in modern security designs. Standardize on **SHA-256**, **SHA-512**, or specialized password KDFs.

1.1 Basic String & Raw Bytes Hashing

```
import hashlib

# 1. Hashing Strings (Must convert str to bytes via .encode())
data = "password123"
md5_hash = hashlib.md5(data.encode()).hexdigest()
sha256_hash = hashlib.sha256(data.encode()).hexdigest()

# 2. Hashing Raw Bytes Directly
raw_bytes = b"        "
sha512_hash = hashlib.sha512(raw_bytes).hexdigest()
```

SECTION 2 — INCREMENTAL & FILE HASHING

For large streams or files, update the hash object incrementally in chunks to keep memory usage low (e.g. 64 KB chunks) instead of loading gigabytes into RAM.

```
import hashlib

def hash_file(filepath, algorithm="sha256"):
    hasher = hashlib.new(algorithm)
    with open(filepath, "rb") as f:
        while chunk := f.read(65536): # 64 KB chunks
            hasher.update(chunk)
    return hasher.hexdigest()

# Usage & Integrity Verification
expected_hash = "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"
actual_hash = hash_file("file.bin", "sha256")

if actual_hash == expected_hash:
    print("[+] Hash matches! Integrity check passed.")
else:
    print("[-] Hash mismatch! File changed or corrupted.")
```

SECTION 3 — PBKDF2 & SECURE RANDOMNESS

3.1 Password-Based Key Derivation (PBKDF2)

Derives strong cryptographic keys from user passwords using a random salt and iteration counter to slow down brute-force attacks.

```
import hashlib, os

salt = os.urandom(16) # Unique, random, non-secret salt stored alongside hash
password = b"SuperSecretPass"

key = hashlib.pbkdf2_hmac(
    hash_name="sha256",
    password=password,
    salt=salt,
    iterations=100000, # High work factor
    dklen=32           # Derived Key Length (32 bytes = 256 bits)
)
print(f"Derived Key: {key.hex()}")
```

3.2 Secure Randomness (`secrets` vs `random`)



Critical Rule: Never use Python's default `random` module for key generation, tokens, or nonces (it uses Mersenne Twister which is predictable). Always use `secrets` or `os.urandom()`.

```
import secrets

token_hex = secrets.token_hex(16) # Hexadecimal string
random_bytes = secrets.token_bytes(32) # Raw cryptographic bytes
url_token = secrets.token_urlsafe(32) # URL-safe Base64 string
```

SECTION 4 — HMAC (HASH-BASED MESSAGE AUTHENTICATION CODE)

HMAC combines a **Secret Key + Hash Function + Message** to provide both **Integrity** and **Authentication**. Unlike raw hashes, an attacker cannot forge an HMAC without knowing the secret key.

```
import hmac, hashlib

key = b"secret-key"
message = b"Important message"

# Generate HMAC
mac = hmac.new(key, message, hashlib.sha256).hexdigest()

# Verify HMAC Safely (Constant-time comparison prevents timing attacks)
if hmac.compare_digest(mac, expected_mac):
    print("[+] Valid MAC and authentic sender")
```

SECTION 5 — SYMMETRIC ENCRYPTION (AES-CBC VS AES-GCM)

AES operates on 128-bit blocks with key sizes of 128, 192, or 256 bits. Requires `pip install pycryptodome`.

5.1 AES-CBC (Cipher Block Chaining) — Unauthenticated

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
import os

key = os.urandom(16)
iv = os.urandom(16) # Initialization Vector

# Encrypt
cipher = AES.new(key, AES.MODE_CBC, iv)
ciphertext = cipher.encrypt(pad(b"Confidential Payload Data", AES.block_size))

# Decrypt
decipher = AES.new(key, AES.MODE_CBC, iv)
plaintext = unpad(decipher.decrypt(ciphertext), AES.block_size)
```

5.2 AES-GCM (Galois/Counter Mode) — AEAD (Recommended)

Provides **Confidentiality + Integrity + Authentication** in one pass.



AES-GCM Critical Rule: Never reuse the same nonce with the same key. Nonce reuse completely destroys security and allows plaintext recovery.

```

from Crypto.Cipher import AES
import os

key = os.urandom(32) # AES-256

# Encrypt & Digest
cipher = AES.new(key, AES.MODE_GCM)
nonce = cipher.nonce
ciphertext, tag = cipher.encrypt_and_digest(b"Secret Message")

# Decrypt & Verify
decipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
try:
    plaintext = decipher.decrypt_and_verify(ciphertext, tag)
    print("Decrypted:", plaintext.decode())
except ValueError:
    print("[-] Authentication failed! Ciphertext or tag was tampered with.")

```

Feature	AES-CBC	AES-GCM
Confidentiality	✓ Yes	✓ Yes
Integrity & Authentication	✗ No (Requires external HMAC)	✓ Yes (Built-in AEAD Tag)
Recommended for new builds	✗ No	✓ Yes

SECTION 6 — ASYMMETRIC CRYPTOGRAPHY (RSA & SIGNATURES)

6.1 Key Generation & RSA-OAEP Encryption

```

from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

# Generate 2048-bit Key Pair
keyPair = RSA.generate(2048)
privKey = keyPair.export_key()
pubKey = keyPair.publickey().export_key()

# Encrypt with Public Key
pub_obj = RSA.import_key(pubKey)
cipher = PKCS1_OAEP.new(pub_obj)
ciphertext = cipher.encrypt(b"Top Secret RSA Message")

# Decrypt with Private Key
priv_obj = RSA.import_key(privKey)
decipher = PKCS1_OAEP.new(priv_obj)
plaintext = decipher.decrypt(ciphertext)

```

6.2 RSA Digital Signatures (RSA-PSS)

```
from Crypto.PublicKey import RSA
from Crypto.Signature import pss
from Crypto.Hash import SHA256

key = RSA.generate(2048)
message = b"Important Message"
h = SHA256.new(message)

# Sign using Private Key
signature = pss.new(key).sign(h)

# Verify using Public Key
pub_key = key.publickey()
h_verify = SHA256.new(message)
try:
    pss.new(pub_key).verify(h_verify, signature)
    print("[+] Digital Signature is valid.")
except ValueError:
    print("[-] Invalid signature!")
```

SECTION 7 — ENCODING VS HASHING VS ENCRYPTION

Technique	Reversible?	Secret Key?	Primary Purpose	Examples
Encoding	✓ Yes	✗ No	Data Representation / Transport	Base64, Hex, URL-Encode
Hashing	✗ No	✗ No	One-Way Fingerprint / Integrity	SHA-256, SHA-512
Encryption	✓ Yes	✓ Yes	Confidentiality	AES-GCM, RSA-OAEP
HMAC	✗ No	✓ Secret Key	Integrity + Authentication	HMAC-SHA256
Digital Signature	Verification	✓ Key Pair	Non-repudiation + Integrity	RSA-PSS, ECDSA

SECTION 8 — CTF SCRIPTING, PWNTOOLS & XOR

Requires `pip install pwntools`. Essential for CTF challenges, socket exploitation, and binary analysis.

8.1 Base64, Hex, Packing & Endianness

```
from pwn import *

# Base64 & Hex Encoding
b64_enc = b64e(b"admin_user")      # -> 'YWRtaW5fdXNlcg=='
b64_dec = b64d("YWRtaW5fdXNlcg==") # -> b'admin_user'
hex_enc = hexen(b"hello")          # -> '68656c6c6f'

# Integer Packing & Unpacking (Little Endian by default)
packed32 = p32(0xdeadbeef)         # -> b'i%p'
unpacked32 = u32(b'i%p')           # -> 0xdeadbeef

# Endian Context
context.endian = "big" # or "little"
```

8.2 XOR Operations in Pwntools

XOR Property: $A \oplus K \oplus K = A$. Repeating key XOR is vulnerable to known-plaintext attacks.

```
from pwn import xor

key = b"MYKEY"
data = b"Longer text to encrypt with repeating key"

ciphertext = xor(data, key)
plaintext = xor(ciphertext, key)
```

8.3 Network Socket Automation (Pwntools Remote)

```
from pwn import *

r = remote("ctf.target.com", 1337)

# Receive & Send Pattern
r.recvuntil(b"Enter hash: ")
r.sendline(b"my_calculated_hash")

banner = r.recvline()
r.interactive() # Gives control back to terminal shell
r.close()
```

SECTION 9 — COMBINATORICS & ITERTOOLS

Useful for key space generation, PIN brute-forcing, and custom password mask creation.

```
import itertools, string

# 1. Product (Cartesian Product / PIN Brute-force: 10^4 = 10,000 combinations)
for pin in itertools.product(string.digits, repeat=4):
    current_pin = "".join(pin) # '0000', '0001', ..., '9999'

# 2. Permutations (Order MATTERS: nPr = n! / (n-r)!)
for p in itertools.permutations("ABC"):
    print("".join(p)) # ABC, ACB, BAC, BCA, CAB, CBA

# 3. Combinations (Order DOES NOT matter: nCr = n! / (r!(n-r)!))
for c in itertools.combinations("ABC", 2):
    print("".join(c)) # AB, AC, BC
```

SECTION 10 — MODULAR ARITHMETIC & RSA MATH SOLVER

10.1 Essential Math Operations

```
import math

# Modular Exponentiation: (base^exp) % mod
res = pow(5, 3, 13) # 5^3 mod 13 = 125 mod 13 = 8

# Modular Multiplicative Inverse: (e * d) % phi = 1
e = 7; phi = 120
d = pow(e, -1, phi) # -> 103 (because (7 * 103) % 120 == 1)

# GCD (Greatest Common Divisor)
gcd_val = math.gcd(7, 120) # -> 1 (Coprime)
```

10.2 Mathematical RSA Solver (CTF / Educational)

```
def solve_rsa(p, q, e, c):
    n = p * q
    phi = (p - 1) * (q - 1)
    d = pow(e, -1, phi) # Private key exponent
    m = pow(c, d, n) # Decrypted integer message

    # Convert Integer to Bytes
    length = (m.bit_length() + 7) // 8
    return m.to_bytes(length, "big")

# Integer <-> Bytes Conversion
num = 123456789
data_bytes = num.to_bytes((num.bit_length() + 7) // 8, "big")
recovered_num = int.from_bytes(data_bytes, "big")
```

SECTION 11 — CONCURRENCY & DICTIONARY TESTING

Use `ThreadPoolExecutor` for I/O-bound tasks (network scanning/requests). For pure CPU hash cracking, dedicated C tools like *hashcat* or *John the Ripper* are far faster due to Python's GIL.

```

import hashlib
from concurrent.futures import ThreadPoolExecutor
from threading import Event

TARGET_HASH = "5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8"
found = Event() # Thread-safe boolean flag

def check_word(word):
    if found.is_set():
        return
    candidate = word.rstrip("
")
    if hashlib.sha256(candidate.encode()).hexdigest() == TARGET_HASH:
        print(f"[+] Password Found: {candidate}")
        found.set()

def main():
    with open("/usr/share/wordlists/rockyou.txt", "r", encoding="latin-1") as f:
        with ThreadPoolExecutor(max_workers=20) as executor:
            executor.map(check_word, f)

if __name__ == "__main__":
    main()

```

SECTION 12 — SYSTEM & NETWORKING QUICK-REFERENCE

12.1 Subprocess (Prevent Command Injection)

```

import subprocess

# SAFE: Pass arguments as list, shell=False
result = subprocess.run(["whoami"], capture_output=True, text=True, check=True)
print(result.stdout.strip())

# UNSAFE: Avoid subprocess.run(user_input, shell=True)

```

12.2 Raw Socket Connection

```

import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("127.0.0.1", 1337))
s.sendall(b"Hello
")
data = s.recv(1024)
s.close()

```

12.3 Conversion Cheat Table

```
# String -> Bytes:      data.encode()
# Bytes -> String:      data.decode()
# Bytes -> Hex:         data.hex()
# Hex -> Bytes:         bytes.fromhex("68656c6c66")
# Integer -> Bytes:     number.to_bytes(length, 'big')
# Bytes -> Integer:     int.from_bytes(data, 'big')
# Base64 -> Bytes:      base64.b64decode(b64_str)
# Bytes -> Base64:      base64.b64encode(raw_bytes)
```



MASTER GOLDEN RULES OF CRYPTOGRAPHY

1. **Hashing $\neq$ Encryption $\neq$ Encoding.** Hashing is one-way; Encoding is reversible without keys; Encryption requires keys.
2. **Never hard-code secrets** or private keys in source code. Use environment variables.
3. **Never reuse AES-GCM nonces** under the same secret key.
4. **Never use SHA-256 alone for passwords.** Use Argon2, scrypt, bcrypt, or PBKDF2 with salt.
5. **Always use `secrets`** instead of `random` for token and key generation.
6. **Use constant-time comparisons** (`hmac.compare_digest`) to prevent timing attacks.
7. **Don't invent your own crypto algorithms.** Rely on standard libraries (PyCryptodome).